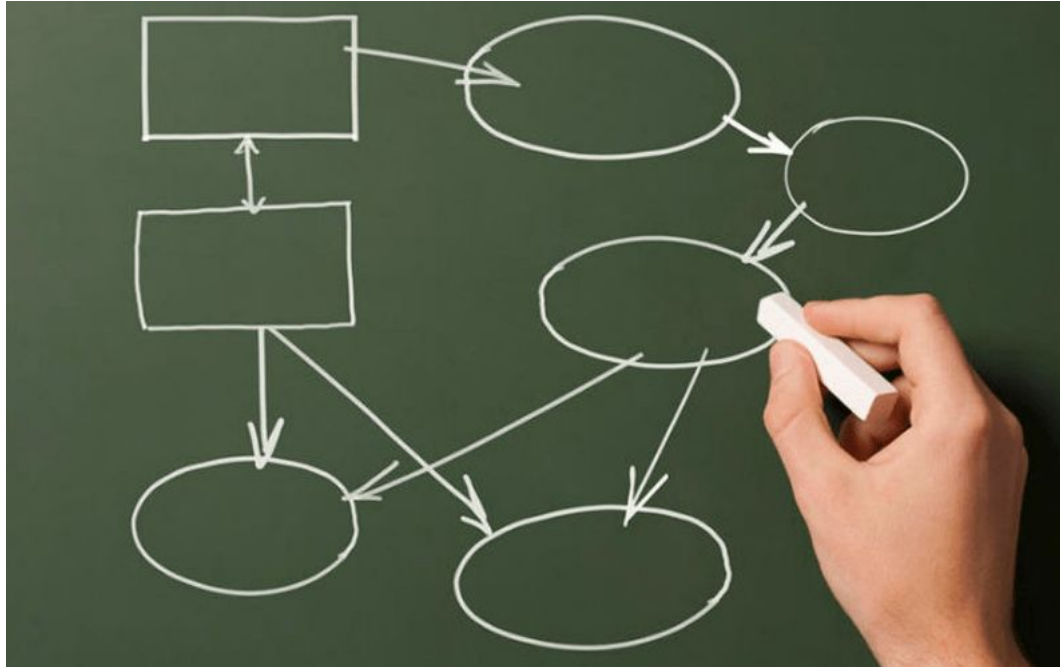
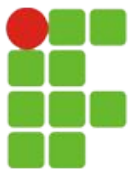
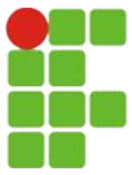




Algoritmos e programação– ALG

Prof. Jones



Recordando

Equações matemáticas

“*sqrt()*”: calcula a raiz quadrada;

“*pow(base, pot)*”: eleva **base** ao **pot** (*potência*);

“*sin()*”: calcula o valor de seno;

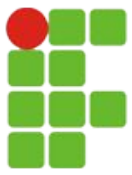
“*cos()*”: calcula o valor de cosseno;

“*tan()*”: calcula o valor da tangente;

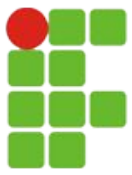
– **Ex.:** *tan(3.14/4);* *//funciona igual a sin() e cos()*

“*log()*”: calcula o valor do logaritmo;

“*log10()*”: calcula o logaritmo na base 10;



Manipulação de cadeias

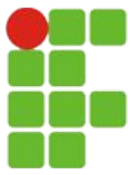


Manipulação de cadeias

Um tipo de dados bastante usado no dia a dia são as **strings**, ou cadeias de caracteres (ou sequências de caracteres). O tipo de dados string, em C++, possui várias operações úteis associadas a ele. Essas operações tornam o C++ uma linguagem bastante propícia para manipulação de textos.

Para manipulação de **strings**, utilizaremos a biblioteca **<string>** do C++. Algumas funções básicas dessa bibliotecas são:

- length(): retorna o tamanho corrente da string;
- strcat(): une duas strings;
- replace(): Compara duas strings;
- Find(): procura se existe uma parte da string;
- To_string(): Converte qualquer tipo primitivo em String



String

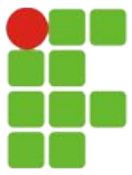
Utilizamos strings para guardar dados não numéricos, com mais de uma letra. Exemplo: Nome de uma pessoa, endereço, Bairro, Nome da cidade, Descrição de um produto, etc....

```
#include <iostream>

using namespace std;

int main() {
    string Nome;
    cout << "Digite o seu nome..: ";
    getline(cin, Nome);
    cout << "O nome digitado foi = " << Nome << endl;
    return 0;
}
```

Para ler uma string usamos o comando **getline**, responsável por pegar uma linha de dados contendo espaços e caracteres



Tamanho da string

Há situações em que precisamos verificar o tamanho da cadeia que foi inserida na variável do tipo string. Para isso usamos o comando **length()**

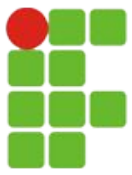
```
#include <iostream>

using namespace std;

int main() {
    int tamanho;
    string Nome;
    cout << "Digite o seu nome..: ";
    getline(cin, Nome);
    tamanho = Nome.length();
    cout << "O tamanho da string = " << tamanho << endl;
    return 0;
}
```

```
Digite o seu nome..: IFSP Barretos
O tamanho da string = 13
```

```
Process returned 0 (0x0)   execution time : 15.400 s
Press any key to continue.
```



Conversão de string para float

As vezes temos um dado armazenado como string que precisa ser convertido em número real para realização de um determinado cálculo. Sabemos que, mesmo com um dado numérico em uma string, não conseguimos realizar operações aritméticas, pois o computador entende como um texto. Para a conversão, utilizamos a função **stof(VARIAVEL)** – String para float

s (string) ↓ f (float)
to (Para)

```
#include <iostream>
#include <iomanip>

using namespace std;

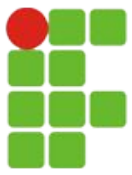
int main() {
    float numero_real;
    string entrada;
    cout << "Digite o a entrada..: ";
    getline(cin, entrada);
    numero_real = stof(entrada);
    cout << "Dado em float = " << fixed << setprecision(2) << numero_real << endl;
    return 0;
}
```

Digite o a entrada..: 123.45678

Dado em float = 123.46

Process returned 0 (0x0) execution time : 15.073 s

Press any key to continue.



String para int

Da mesma forma, podemos converter um dado do tipo string em inteiro (int). Para isso, utilizamos a função **stoi(VARIAVEL)**

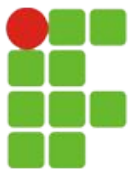
s (string) ↓ i (int)
to (Para)

```
#include <iostream>

using namespace std;

int main() {
    string num1 = "1";
    string num2 = "2";
    cout << stoi(num1) + stoi(num2);
    return 0;
}
```

```
3
Process returned 0 (0x0)   execution time : 0.036 s
Press any key to continue.
```



String para double

Podemos também converter uma string em double, usando a função **stod(VARIAVEL)**

s (string) → d (double)
to (Para)

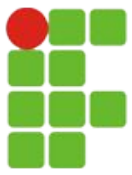
```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    double numero_double;
    string entrada;
    cout << "Digite o a entrada..: ";
    getline(cin, entrada);
    numero_double = stod(entrada);
    cout << "Dado em double = "
         << fixed
         << setprecision(2)
         << numero_double << endl;
    return 0;
}
```

```
Digite o a entrada..: 64.123456
Dado em double = 64.12
```

```
Process returned 0 (0x0)   execution time : 3.198 s
Press any key to continue.
```



bool/float/int/double para String

Podemos também converter qualquer tipo primitivo em string, usando a função **to_string (VARIÁVEL)**

to (Para)

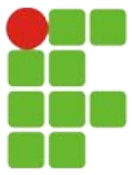
```
#include <iostream>

using namespace std;

int main() {
    int a = 10;
    double b = 20.0;
    float c = 20.2020;
    bool d = true;
    cout << to_string(a) << endl;
    cout << to_string(b) << endl;
    cout << to_string(c) << endl;
    cout << to_string(d) << endl;
    return 0;
}
```

```
10
20.000000
20.202000
1
```

```
Process returned 0 (0x0)   execution time : 0.033 s
Press any key to continue.
```



Maiúsculo e minúsculo

É possível converter os caracteres de uma string em **MAIÚSCULO** ou **MINÚSCULO**. A função para transformar **(transform)** as letras em maiúsculo é **toupper**, e para minúsculo **tolower**. Essas funções estão dentro da biblioteca **<algorithm>** do C++.

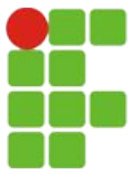
```
#include <iostream>
#include <algorithm>
```

```
using namespace std;
```

```
int main() {
    string entrada = "Hello World";
    transform(entrada.begin(), entrada.end(), entrada.begin(), ::tolower);
    cout << entrada << endl;
    transform(entrada.begin(), entrada.end(), entrada.begin(), ::toupper);
    cout << entrada << endl;
    return 0;
}
```

```
hello world
HELLO WORLD
```

```
Process returned 0 (0x0)   execution time : 0.037 s
Press any key to continue.
```



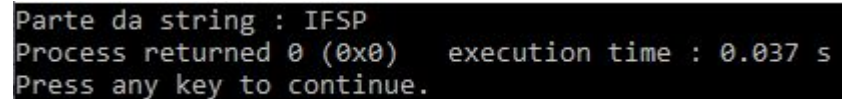
Subtring

Existem situações que dado uma string precisamos pegar apenas uma parte dela. Para isso utilizamos o comando **substr(INICIO, COMPRIMENTO)**.

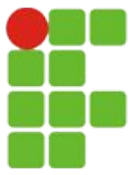
```
#include <iostream>

using namespace std;
```

```
int main() {
    string entrada = "Eu estudo no IFSP Barretos";
    string parte = "";
    parte = entrada.substr(13, 4);
    cout<<"Parte da string : "<< parte;
    return 0;
}
```



```
Parte da string : IFSP
Process returned 0 (0x0)   execution time : 0.037 s
Press any key to continue.
```



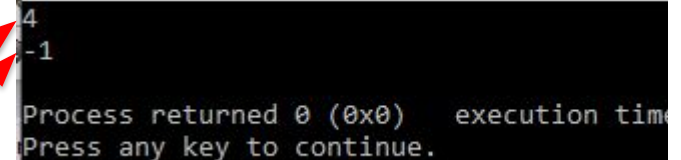
Find

De forma análoga, podemos verificar se existe uma subparte dentro da nossa string. Exemplo, na string “ALO MUNDO”, queremos saber se existe a palavra “MUNDO”. Para isso, utilizamos a função **find()**. Esta função retorna **-1** caso não tenha encontrado uma palavra dentro da string, ou a **posição** onde ele encontrou aquela palavra.

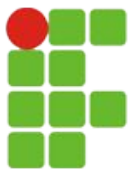
```
#include <iostream>

using namespace std;

int main() {
    string entrada = "ALO MUNDO";
    int r1 = entrada.find("MUNDO");
    cout << r1 << endl;
    int r2 = entrada.find("IFSP");
    cout << r2 << endl;
    return 0;
}
```



```
4
-1
Process returned 0 (0x0) execution time:
Press any key to continue.
```



Replace

Podemos substituir uma parte da nossa string por outro valor. Para isso, utilizamos a função **replace()**. No exemplo abaixo, trocamos a palavra **MUNDO** por **BARRETOS**.

```
#include <iostream>

using namespace std;

int main() {
    string entrada = "ALO MUNDO";
    entrada.replace(4, 8, "BARRETOS");
    cout << entrada << endl;
    return 0;
}
```

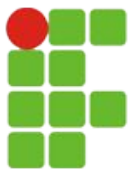
A partir de qual caractere quero trocar?

Nº de caracteres que serão substituídas?

Nova cadeia ou palavra

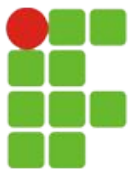
ALO BARRETOS

Process returned 0 (0x0) execution time : 0.054 s
Press any key to continue.



Replace, outro exemplo

```
#include <string>
int main()
{
    string cad("Uva, banana, laranja");
    cout << cad << endl;
    // "Uva, banana, laranja"
    cad.replace(5, 8, "abacaxi, pera, ");
    // "Uva, abacaxi, pera, laranja"
    cout << cad << endl;
    return 0;
}
```



Comparando strings

Há problemas em que devemos analisar se duas strings são similares, ou seja, se contém o mesmo conteúdo. Para isso, utilizamos a função **compare()**. Esta função retorna **0** se as duas strings são **iguais**, e **1** se **não são similares**.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string entrada1 = "IFSP";
    string entrada2 = "IFSP";
    int retorno = entrada1.compare(entrada2);
    cout << retorno << endl;
    return 0;
}
```

0

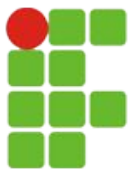
Process returned 0 (0x0) execution time : 0.027 s
Press any key to continue.

```
#include <iostream>
#include <string>
using namespace std;

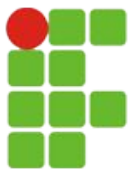
int main() {
    string entrada1 = "MUNDO";
    string entrada2 = "IFSP";
    int retorno = entrada1.compare(entrada2);
    cout << retorno << endl;
    return 0;
}
```

1

Process returned 0 (0x0) execution time : 0.153 s
Press any key to continue.



Conversão de bases



Conversão de base

Na linguagem C++ há funções que fazem a conversão de bases numéricas.

Decimal para binário (Vice versa)

Decimal para octal (Vice versa)

Decimal para hexadecimal (Vice versa)

```
#include <iostream>

using namespace std;

int main()
{
    int a = 10;
    char res[50];
    itoa(a, res, 2);
    cout<<"Hexadecimal- "<< hex << a << endl;
    cout<<"Decimal- "<< dec << a << endl;
    cout<<"Octal- "<< oct << a << endl;
    cout<<"Binario- "<< res << endl;
    return 0;
}
```



Conversão de decimal para binário



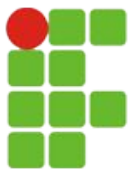
Conversão de decimal para hexadecimal



Conversão de decimal para decimal (poderia ser outra base)



Conversão de decimal para octal



Hexadecimal em Maiúsculo ou minúsculo

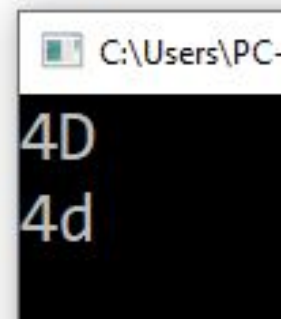
Nós podemos também deixar a saída do **cout** em letras maiúsculas ou minúsculas. Basta utilizarmos as funções **uppercase** e **nouppercase** antes do texto ou conteúdo de variáveis.

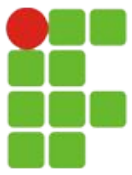
```
#include <iostream>
#include <string>

using namespace std;

int main()
{
    cout << hex << uppercase << 77 << endl;
    cout << hex << nouppercase << 77 << endl;

    return 0;
}
```





Conversão de base

Binário p/ decimal
hexa p/ decimal
octal p/ decimal

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int main() {
```

```
    string h = "1A";
```

```
    string b = "11";
```

```
    string o = "31";
```

```
    int hexa = stoi(h, 0, 16);
```

```
    int bin = stoi(b, 0, 2);
```

```
    int octal = stoi(o, 0, 8);
```

```
    cout << hexa << endl;
```

```
    cout << bin << endl;
```

```
    cout << octal << endl;
```

```
    return 0;
```

```
}
```

- Conversão de hexa para decimal
- Conversão de binário para decimal
- Conversão de hexa para hexadecimal

26
3
25

Problema G

Criptografia

Por Jones Mendonça de Souza (IFSP – campus Barretos)

Arquivo: *criptografia.[c|cpp|java]*

Timelimit: 1

Algoritmos

Exercícios

A criptografia é um conjunto de técnicas pensadas para proteger uma informação de modo que apenas o emissor e o receptor consigam compreendê-la. A Agência de Segurança Nacional (NSA) dos Estados Unidos pretende construir um algoritmo de criptografia para realizar a comunicação entre seus soldados durante uma possível guerra. O objetivo do algoritmo é criptografar as coordenadas cartesianas dos pontos X e Y do território inimigo e, enviar esses dados até a sua base local, que irá proceder com lançamentos de mísseis. O procedimento de codificação é muito simples, para cada coordenada some o antecessor com o sucessor da coordenada correspondente e o resultado que se encontra na base decimal converta-o para base hexadecimal, considere o exemplo abaixo:

Coordenada X: 85	Coordenada Y: 466
$84 + 86 = 170$	$465 + 467 = 932$
Coordenada X convertida em hexadecimal: AA	Coordenada Y convertida em hexadecimal: 3A4

Entrada

A entrada contém 2 valores inteiros, o primeiro valor corresponde a coordenada X e o segundo a coordenada Y.

Saída

Para cada entrada, deve-se apresentar as coordenadas X e Y criptografadas. Imprima a mensagem da seguinte maneira:

"X = " (letra X maiúscula) seguido pelo valor criptografado

"Y = " (letra Y maiúscula) seguido pelo valor criptografado.

Cuide para que tenha um espaço antes e depois do sinal de igualdade e para que imprima o fim de linha após o resultado, conforme exemplos abaixo.

Exemplos de Entradas	Exemplos de Saídas
85 466	X = AA Y = 3A4
106 224	X = D4 Y = 1C0
99 423	X = C6 Y = 34E
66 181	X = 84 Y = 1E8

Exercícios

- 1) Escreva um programa em C++ para apontar se a palavra digitada pelo usuário está no gerúndio “ando”. Por exemplo, a palavra “Estudando” está no gerúndio. Mostre -1 se o resultado for negativo ou a posição do gerúndio.
- 2) Faça um programa que destaque o gerúndio de uma palavra digitada pelo usuário em letras maiúsculas. Exemplo, a palavra digitada foi “estudando”, mostra “estudANDO”
- 3) Faça um programa em que o usuário informará a data de nascimento no formato DD/MM/AAAA. Mostre um por linha, o dia, o mês e ano. Exemplo: 23/12/1998, mostrar:
Dia = 23
Mês = 12
Ano = 1998
- 4) Faça um programa em que o usuário informará a data de nascimento no formato DD/MM/AAAA. Mostre a idade do usuário.
- 5) Faça um programa em que o usuário irá digitar uma expressão matemática de soma com 3 números. Exemplo: 10+40+50. Seu programa deve calcular essa expressão. Considere todos os número com 2 dígitos.